TRUSTING TRUST REVISITED

Implementing and Detecting a Self Replicating Compiler Backdoor in C++

A Project Report

Submitted in partial fulfilment of the requirements of
the Compiler Design course

Domain	Compiler Design and Software Supply Chain Security
Implementation Language	C++ (C++17)
Project Type	Individual project
Report Date	September 2026

Abstract

In 1984 Ken Thompson described an attack in which a compiler is modified to recognize the program it is compiling and insert malicious logic that never appears in the corresponding source code. When the compiler additionally recognizes its own source and reinserts this logic into every future version of itself, the malicious behaviour persists indefinitely even after the original modification is deleted from the visible source tree. This report documents a complete, working, ground up implementation of this attack and of David A. Wheeler's Diverse Double Compiling defence, built around a small TinyLang to C++ transpiler written from scratch in C++17. The implementation is safety scoped throughout: the payload the compiler injects is an inert, clearly labelled marker rather than a functional credential bypass, while the detection, injection, self recognition, self replication, and defence mechanisms are all fully real and were verified experimentally, including a confirmed stable fixed point across three successive compiler generations and a working Diverse Double Compiling harness that correctly distinguishes an infected build from a clean one. The report situates this work against the existing literature on trusting trust attacks and reproducible builds, identifies the specific gap it addresses, and assesses its novelty on a numeric scale with justification.

1. Problem Identification

A compiler occupies a privileged position in the software development process. Every review of an application's source code implicitly assumes that the compiler used to build it is a faithful, behaviour preserving translator. Thompson showed that this assumption can be violated in a way that source level review cannot detect, because the malicious behaviour can be made to live entirely inside a compiled binary rather than in any file a human ever reads. This raises two concrete problems that motivate this project.

1. Demonstrability. The trusting trust attack is widely cited but rarely implemented end to end by students, because a full implementation requires solving several nontrivial subproblems together: building a real compiler front end and back end, constructing a self reproducing program in a language where this is notoriously awkward, and combining the two so that the self reproduction carries a targeted modification forward indefinitely. Without a working artifact, the attack remains an abstract idea rather than a demonstrated mechanism.
2. Verifiability of defences. Diverse Double Compiling is frequently mentioned as the accepted countermeasure, but its effectiveness is rarely shown against a real, self replicating infected compiler built by the same student presenting the defence. A defence that is only described, and never tested against a genuine positive case and a genuine negative case, does not establish that it actually discriminates infected from clean builds.

This project treats both problems as the thing to be solved: build a real, working instance of the attack, and build a real, working instance of the defence, and verify both experimentally rather than by description alone.

2. Literature Survey and Existing Work

2.1 Foundational work

The trusting trust attack was first identified by Karger and Schell in a 1974 United States Air Force security evaluation of Multics, and was later independently rediscovered, implemented, and popularised by Thompson in his 1984 Turing Award lecture, which remains the canonical reference for the attack. Thompson's lecture is conceptual and does not include a runnable implementation; it establishes the idea in prose and a short

illustrative code fragment rather than a complete toolchain.

2.2 The Diverse Double Compiling defence

Wheeler's 2005 conference paper and 2009 doctoral dissertation introduce Diverse Double Compiling as a practical technique to detect a trusting trust style compromise without needing to read or trust the suspect compiler's source. The dissertation generalises the 2005 result, treats the original scheme as a special case termed self regeneration, and includes a formal proof together with experiments performed against GCC. Wheeler has continued to write about the technique in the years since, connecting it explicitly to the broader Reproducible Builds effort.

2.3 Later implementations and extensions

Skrimstad's 2018 master's thesis, Improving Trust in Software through Diverse Double Compiling and Reproducible Builds, implements a self replicating compiler attack against the Go compiler and evaluates Diverse Double Compiling against it, making it the closest prior work to this project in spirit, though it targets a different language and toolchain and does not use a safety scoped payload. More recent work has shown that the general trusting trust idea is not limited to compilers: a 2025 paper constructs a complete trusting trust style attack using GNU strip, an ordinary build utility that neither parses nor generates source code, demonstrating that the same self perpetuating structure can be built around binary manipulation alone. A 2024 to 2025 KTH master's thesis examines Diverse Double Compiling specifically in the context of modern continuous integration and continuous delivery pipelines.

2.4 Real world relevance

The trusting trust attack is not merely historical. The March 2024 discovery of a backdoor in XZ Utils, tracked as CVE 2024 3094, showed a build time supply chain compromise of comparable severity in widely deployed Linux infrastructure, where obfuscated logic hidden in test fixtures modified the build process to inject a backdoor into the resulting liblzma library. The incident is frequently cited in the Reproducible Builds community as a concrete illustration of why build process integrity, and not only source code review, matters. In response, efforts such as the Reproducible Builds project, the Bootstrappable Builds and GNU Mes projects, and FreeBSD's Zero Trust Builds initiative, which reached its development branch in 2025, aim to reduce or eliminate implicit trust in build toolchains at an infrastructure scale.

2.5 Gap addressed by this project

The existing literature separates cleanly into conceptual description (Thompson), formal defence with limited public tooling (Wheeler), and language specific research implementations aimed at an academic or security research audience (Skrimstad, the GNU strip paper) that are not designed as a self contained teaching artifact. None of the surveyed prior work presents, in a single small C++ codebase suitable for a compiler design course, all of the following together: a working lexer, parser and code generator built from scratch, a verified C++ quine built using raw string literals, the quine technique generalised to carry two independent payloads through a self replicating compiler, an explicit engineering record of a genuine bug found and fixed during development, a safety scoped payload design with an explicit rationale, and a working Diverse Double Compiling harness verified against both a positive and a negative case using artifacts from the same project. This combination is the specific gap this project addresses.

3. Proposed Work

The proposed work implements the attack and its defence in five phases, each independently buildable and independently verified.

Phase	Description	Verification performed
1	A deterministic TinyLang to C++ compiler: DFA based lexer, recursive descent parser, code generator, and a tree walking interpreter.	Unit tests for the lexer and parser, golden file tests comparing generated C++ output, and end to end tests that compile and run the generated programs.
2	A targeted, inert marker injected into the generated code only when a chosen trigger phrase appears in the compiler's own source.	Confirmed the marker appears only for the target compilation and not for ordinary compilations.
3	A C++ quine built with raw string literals, resolving the delimiter problem that makes self reproduction non trivial.	Output compared byte for byte against the source, and compiled and executed to confirm it reproduces itself across two succes
4	Self recognition of the compiler's own source and self replication of the injection logic into freshly regenerated compilers.	A tainted compiler was produced from a clean one, then used to build a further compiler from a modified source, and that was proc
5	A Diverse Double Compiling harness comparing a trusted clean compiler against the self hosted chain.	Ran the harness end to end, checking for the infected compiler and correctly

A single deliberate scoping decision governs Phases 2 and 4: the payload the compiler injects is a labelled, inert marker rather than a functional authentication bypass. The injection mechanism, the self recognition logic, and the self replication technique are all implemented for real and behave exactly as the historical attack describes. Only the content of the payload is changed, which preserves every structural property the project is meant to demonstrate while keeping the resulting artifact safe to build, run and submit.

4. Novelty and Novelty Score

This project does not claim to introduce a new attack, a new defence, or a new theoretical result. The underlying attack was described forty years before this report and the defending technique was described twenty years before this report, and both are established, published, peer reviewed work. An honest novelty assessment must therefore separate conceptual novelty, which is low, from implementation and pedagogical novelty, which is the actual contribution of this project.

4.1 Novelty score

Dimension	Score out of 10	Justification
Conceptual novelty	1	The attack and the defence are both established results from the existing literature
Implementation novelty	6	No surveyed prior work combines a from scratch lexer, parser and code generator,
Pedagogical and safety design novelty	7	The explicit substitution of an inert, clearly labelled payload for a functional credent
Overall novelty score	**5**	A weighted overall assessment, reflecting genuine but modest originality concentra

Overall novelty score: 5 out of 10. This reflects a project that is not original at the level of ideas, but is original at the level of a complete, verified, safety conscious implementation suitable for use as a teaching artifact, which is a meaningful and defensible contribution for coursework of this kind without overstating what has been achieved.

5. Objectives

1. To design and implement a minimal but genuine compiler front end and back end, including a deterministic finite automaton based lexer, a recursive descent parser producing an explicit abstract syntax tree, and a code generator, for a small purpose built language.
2. To implement a targeted code injection mechanism inside the code generator that inserts logic a program's source code never requested, and to verify experimentally that the injection is correctly targeted and does not alter unrelated program behaviour.
3. To construct and verify a self reproducing C++ program, resolving the raw string literal delimiter collision problem that this language presents for quines.
4. To combine self recognition and self reproduction so that a compiler detects when it is processing its own source and regenerates its injection logic into the new source it produces, and to verify that this survives repeated compilation across multiple generations.
5. To implement and verify a Diverse Double Compiling harness capable of distinguishing an infected compiler lineage from a clean one using only externally observable build outputs.
6. To document the project in a manner suitable for presentation, including an honest, evidence based assessment of its novelty relative to the existing literature.

6. Expected Outcomes and Deliverables

1. A working TinyLang to C++ compiler with an automated test suite.
2. A demonstrated, reproducible instance of targeted code injection.
3. A verified C++ quine and an accompanying explanation of the delimiter collision problem it solves.
4. A working self replicating compiler whose infection is shown to reach a stable fixed point across at least three generations.
5. A working Diverse Double Compiling harness verified against both a positive case, an infected compiler, and a negative case, a clean baseline pair.
6. A written report, of which this document is the primary artifact, situating the implementation against the existing literature and assessing its novelty on a justified numeric scale.

7. Conclusion

This project does not seek to advance the theory of trusting trust attacks or their defences, both of which are settled in the existing literature. Its contribution is a complete, verified, safety scoped implementation that makes the mechanism and its defence directly observable rather than only described, built specifically for use as a compiler design coursework artifact. The strongest evidence for the project's value is experimental rather than theoretical: every phase was actually built, actually run, and actually checked against a defined pass or fail criterion before being considered complete, including a genuine defect that was found and corrected during development of the self replication mechanism.

References

1. K. Thompson, Reflections on trusting trust, Communications of the ACM, volume 27, issue 8, pages 761 to 763, 1984.

2. P. A. Karger and R. R. Schell, Multics Security Evaluation, Vulnerability Analysis, ESD TR 74 193, United States Air Force Electronic Systems Division, 1974.

3. D. A. Wheeler, Countering Trusting Trust through Diverse Double Compiling, Proceedings of the 21st Annual Computer Security Applications Conference, IEEE Computer Society, pages 33 to 48, 2005.

4. D. A. Wheeler, Fully Countering Trusting Trust through Diverse Double Compiling, Doctor of Philosophy dissertation, George Mason University, 2009.

5. Y. Skrimstad, Improving Trust in Software through Diverse Double Compiling and Reproducible Builds, Master's thesis, 2018.

6. Author collective, A Trusting Trust Attack against an Entire Linux Distribution through Binary Manipulation using GNU strip, security research publication, 2025.

7. Zscaler ThreatLabz, Snyk, and CrowdStrike security advisories on CVE 2024 3094, the XZ Utils supply chain backdoor, published April 2024.

8. Reproducible Builds project, monthly report for October 2025, reproducible builds dot org, including coverage of FreeBSD Zero Trust Builds.

9. D. A. Wheeler, supporter interview on supply chain security, Reproducible Builds project, December 2022.

10. KTH Royal Institute of Technology, Diverse Double Compiling in a Continuous Integration and Continuous Delivery Pipeline, master's thesis, diva portal, 2024 to 2025.

11. GNU Mes project and the Bootstrappable Builds initiative, project documentation on reducing trusted computing base size during compiler bootstrap.

Note on scope. All source code, build scripts, and verification logs referenced in this report accompany it as a separate project archive organised by phase.